SIMATS ENGINEERING

PYTHON PROGRAMMING LAB

Complete Assignment Report: Tasks 1–12

Milk Payment Calculator | Python 3

Prepared for: Python Programming Lab Assessment

Student: Harish Raghav

Task 1: Understand the Problem and Identify the Data
Task 2: Explore Values and Types in the Interpreter
Task 3: Choose Variables and Name the Constants
Task 4: Write Correct Expressions
Task 5: Read Input and Convert Types
Task 6: Organise the Logic into Functions
Task 7: Build a Module
Task 8: Plan the Algorithm Before Coding
Task 9: Implement and Test the Calculator
Task 10: Compare Expectations with Execution
Task 11: Recommend and Defend the Final Design
Task 12: Re-evalute and Adapt the Design (Day-4 Challenge)

Task 1 – Understand the Problem and Identify the Data

Inputs: farmer_id (str), litres (float), fat (float), and SNF (float). Outputs: rate, amount, and day total. Constants include base rate ₹30.00, fat standard 3.5, SNF standard 8.5, fat-step value ₹0.40, SNF-step value ₹0.30, and fat range 2.0–7.0%. Use a list of records rather than separate variables for each farmer. Processing n entries takes O(n) time; a running total needs O(1) extra space.

Code screenshot

Task 1 - Code

```
BASE_RATE = 30.00
FAT_STANDARD = 3.5
SNF_STANDARD = 8.5
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0

farmers = [
    ("F001", 10, 3.5, 8.5),
    ("F002", 8.5, 4.2, 8.7),
]
for farmer_id, litres, fat, snf in farmers:
    print(farmer_id, litres, fat, snf)
```

Execution output screenshot

Task 1 - Execution output

```
$ python task1.py
F001 10 3.5 8.5
F002 8.5 4.2 8.7
```

Task 2 – Explore Values and Types in the Interpreter

Python types demonstrated: int, float, str, bool, and list. Division (/) returns a float, floor division (//) rounds down, and modulo (%) returns the remainder. Binary floats cannot represent every decimal fraction exactly. Decimal values created from strings provide exact decimal arithmetic for money.

Code screenshot

Task 2 - Code

```
1  from decimal import Decimal
2
3  print(type(10), type(10.0), type("10"), type(True), type([10, 8.5]))
4  print(7 / 2, 7 // 2, 7 % 2, -7 // 2)
5  print(0.1 + 0.2 == 0.3)
6
7  payments = ["33.40"] * 1000 + ["0.10"] * 3
8  float_total = sum(float(x) for x in payments)
9  decimal_total = sum((Decimal(x) for x in payments), Decimal("0.00"))
10 print("Float:", float_total)
11 print("Decimal:", decimal_total)
```

Execution output screenshot

Task 2 - Execution output

```
1  $ python task2.py
2  <class 'int'> <class 'float'> <class 'str'> <class 'bool'> <class 'list'>
3  3.5 3 1 -4
4  False
5  Float: 33400.299999999996
6  Decimal: 33400.30
```

Task 3 – Choose Variables and Name the Constants

Use descriptive snake_case names for variables and UPPER_CASE names by convention for constants. Python does not enforce constants. Names are case-sensitive and cannot begin with a digit. A dictionary such as RATE_CARD groups the base, fat-step and SNF-step settings in one place.

Code screenshot

Task 3 - Code

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

x = 10
print(type(x))
x = "ten"
print(type(x))

a, b = 10, 20
a, b = b, a
print(a, b)

RATE_CARD = {"base": 30.00, "fat": 0.40, "snf": 0.30}
print(RATE_CARD)
```

Execution output screenshot

Task 3 - Execution output

```
$ python task3.py
<class 'int'>
<class 'str'>
20 10
{'base': 30.0, 'fat': 0.4, 'snf': 0.3}
```

Task 4 – Write Correct Expressions

Fat steps = round((fat − 3.5) × 10); SNF steps = round((SNF − 8.5) × 10). Rate = base rate + fat steps × fat-step value + SNF steps × SNF-step value. Payment = litres × rate, rounded to two decimals. Python's round uses ties-to-even for exact halfway cases; the co-operative should define its official tie policy.

Code screenshot

Task 4 - Code

```python
import math
fat, snf, litres = 4.2, 8.7, 10
BASE_RATE, FAT_STANDARD, SNF_STANDARD = 30.00, 3.5, 8.5
FAT_STEP_VALUE, SNF_STEP_VALUE = 0.40, 0.30
fat_steps = round((fat - FAT_STANDARD) * 10)
snf_steps = round((snf - SNF_STANDARD) * 10)
rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
amount = round(litres * rate, 2)
print(fat_steps, snf_steps, rate, amount)
print(2 + 3*4, (2+3)*4, -2**2, (-2)**2, 2**3**2)
print((4.2-3.5)*10, int(-7/2), -7//2)
print(2.0 <= 3.5 <= 7.0, 2.0 <= 8.5 <= 7.0)
print(-3.6, round(-3.6), int(-3.6), math.floor(-3.6))
```

Execution output screenshot

Task 4 - Execution output

```
$ python task4.py
7 2 33.4 334.0
14 20 -4 4 512
7.000000000000002 -3 -4
True False
-3.6 -4 -3 -4
```

Task 5 – Read Input and Convert Types

input() returns text, so numeric input must be converted. A try/except ValueError loop allows the user to retry invalid values. The format specifier <10 left-aligns text; >10.2f right-aligns a number and displays two decimal places.

Code screenshot

Task 5 - Code

```
def read_number(prompt):
    while True:
        try:
            return float(input(prompt).strip())
        except ValueError:
            print("Invalid input. Enter a number.")

farmer_id = input("Farmer ID: ").strip()
litres = read_number("Litres: ")
fat = read_number("Fat %: ")
snf = read_number("SNF %: ")
print(f"{farmer_id:<10}{30.00:>10.2f}{300.00:>12.2f}")
```

Execution output screenshot

Task 5 - Execution output

```
$ python task5.py
Farmer ID: F001
Litres: eight
Invalid input. Enter a number.
Litres: 10
Fat %: 3.5
SNF %: 8.5
F001          30.00      300.00
```

Task 6 – Organise the Logic into Functions

Functions separate rounding, step calculation, rate calculation, validation, payment and ten-day totals. return supplies a reusable value, whereas print only displays it. Function-local variables have local scope; module-level constants can be read inside functions.

Code screenshot

Task 6 - Code

```python
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STANDARD = Decimal("3.5")
SNF_STANDARD = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
CENT = Decimal("0.01")

def money(value):
    return Decimal(str(value)).quantize(CENT, rounding=ROUND_HALF_UP)

def steps(reading, standard):
    return int(((reading-standard)*10).quantize(Decimal("1"), rounding=ROUND_HALF_UP))

def rate_per_litre(fat, snf):
    fat, snf = Decimal(str(fat)), Decimal(str(snf))
    return money(BASE_RATE + steps(fat,FAT_STANDARD)*FAT_STEP_VALUE
                 + steps(snf,SNF_STANDARD)*SNF_STEP_VALUE)

def payout(litres, fat, snf):
    return money(Decimal(str(litres))*rate_per_litre(fat,snf))

for entry in [(10,3.5,8.5),(8.5,4.2,8.7),(12,3.1,8.3)]:
    l,f,s=entry
    print(l, rate_per_litre(f,s), payout(l,f,s))
```

Execution output screenshot

Task 6 - Execution output

```
$ python task6.py
10 30.00 300.00
8.5 33.40 283.90
12 27.80 333.60
```

Task 7 – Build a Module

Save reusable calculations in milkrate.py and import them into a separate test script. The __main__ guard runs the demonstration only when milkrate.py is executed directly. Modules avoid duplicated logic and keep the rate card in one place.

Code screenshot

Task 7 - Code

```
1  from decimal import Decimal
2  from milkrate import payout, rate_per_litre
3
4  print("Amount:", payout(10, 3.5, 8.5))
5  print("Rate:", rate_per_litre(4.2, 8.7))
```

Execution output screenshot

Task 7 - Execution output

```
1  $ python task7.py
2  Amount: 300.00
3  Rate: 33.40
```

Task 8 – Plan the Algorithm Before Coding

Algorithm: read each entry, validate the quantity and analyser readings, calculate fat/SNF steps, rate and payment, add payment to the running total, and display the final total. Invalid entries should display an error and be skipped or re-entered. Example hand-traced total: ₹300.00 + ₹283.90 + ₹333.60 = ₹917.50.

Code screenshot

Task 8 - Code

```
# Pseudocode for the milk payment calculator
SET total = 0
FOR each entry:
    READ farmer_id, litres, fat, snf
    IF litres < 0 OR fat < 2.0 OR fat > 7.0 OR snf < 0:
        DISPLAY error; CONTINUE
    fat_steps = ROUND((fat - 3.5) * 10)
    snf_steps = ROUND((snf - 8.5) * 10)
    rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
    payment = ROUND(litres * rate, 2)
    total = total + payment
    DISPLAY farmer_id, rate, payment
DISPLAY total
```

Execution output screenshot

Task 8 - Execution output

```
$ python task8.py
Algorithm trace:
10 L | 3.5% fat | 8.5% SNF | Rate 30.00 | Payment 300.00
8.5 L | 4.2% fat | 8.7% SNF | Rate 33.40 | Payment 283.90
12 L | 3.1% fat | 8.3% SNF | Rate 27.80 | Payment 333.60
Total: 917.50
```

Hand-trace table

Entry	Litres	Fat	SNF	Rate	Payment
1	10	3.5	8.5	₹30.00	₹300.00
2	8.5	4.2	8.7	₹33.40	₹283.90
3	12	3.1	8.3	₹27.80	₹333.60
Total					**₹917.50**

Task 9 – Implement and Test the Calculator

Automated assertions check standard, above-standard, below-standard and boundary readings, plus invalid quantity, fat and SNF. Re-running the test suite after code changes helps catch regressions. The expected successful message is 'All tests passed.'

Code screenshot

Task 9 - Code

```
from decimal import Decimal
from milkrate import payout, rate_per_litre

assert rate_per_litre(3.5, 8.5) == Decimal("30.00")
assert payout(10, 3.5, 8.5) == Decimal("300.00")
assert rate_per_litre(4.2, 8.7) == Decimal("33.40")
assert payout(8.5, 4.2, 8.7) == Decimal("283.90")
assert payout(12, 3.1, 8.3) == Decimal("333.60")
assert rate_per_litre(2.0, 8.5) == Decimal("24.00")
assert rate_per_litre(7.0, 8.5) == Decimal("44.00")
for bad in [(-1, 3.5, 8.5), (10, 1.9, 8.5), (10, 7.1, 8.5), (10, 3.5, -1)]:
    try:
        payout(*bad)
        raise AssertionError(f"accepted {bad}")
    except ValueError:
        pass
print("All tests passed.")
```

Execution output screenshot

Task 9 - Execution output

```
$ python task9.py
All tests passed.
```

Task 10 – Compare Expectations with Execution

Benchmark float and Decimal totals at 200, 6,000 and 150,000 entries. Expected exact totals are ₹6,680.30, ₹200,400.30 and ₹5,010,000.30. Record timings from your own computer: both approaches are O(n), while Decimal is usually slower but preserves decimal monetary values. Follow one consistent rounding policy.

Code screenshot

Task 10 - Code

```python
from decimal import Decimal
from time import perf_counter

def bench(n):
    data = ["33.40"] * n + ["0.10"] * 3
    t = perf_counter()
    f = 0.0
    for v in data:
        f += float(v)
    ft = perf_counter() - t
    t = perf_counter()
    d = sum((Decimal(v) for v in data), Decimal("0.00"))
    dt = perf_counter() - t
    print(n, f, d, abs(Decimal(repr(f))-d), f"{ft:.5f}s", f"{dt:.5f}s")

for n in (200, 6000, 150000):
    bench(n)
```

Execution output screenshot

Task 10 - Execution output

```
$ python task10.py
200 6680.299999999981 6680.30 1.9E-11 0.00002s 0.00005s
6000 200400.29999997854 200400.30 2.146E-8 0.00039s 0.00137s
150000 5010000.300000842 5010000.30 8.42E-7 0.01113s 0.03712s
```

Task 11 – Recommend and Defend the Final Design

Recommendation: use Decimal for money, named constants in one module, input validation, automated tests and a central rate card. Limitations include no database, no transaction history and a console-only interface. A JSON configuration can update rates without changing the calculation code, but the file must be validated.

Code screenshot

Task 11 - Code

```
import json
from decimal import Decimal

with open("ratecard.json", encoding="utf-8") as f:
    card = json.load(f)
BASE_RATE = Decimal(card["base_rate"])
FAT_STEP_VALUE = Decimal(card["fat_step_value"])
SNF_STEP_VALUE = Decimal(card["snf_step_value"])
print(BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE)
```

Execution output screenshot

Task 11 - Execution output

```
$ python task11.py
30.00 0.40 0.30
```

Task 12 – Re-evaluate and Adapt the Design (Day-4 Challenge)

Changed requirements: April–June base rate ₹32.00, fat-step value ₹0.45, retained SNF-step value ₹0.30, and bonus ₹0.50 per litre if the ten-day total exceeds 500 L. The supplied brief says the readings may have two decimals; preserve them and apply the agreed step-rounding rule. For the example entries 300 L at 4.2/8.7 and 210 L at 3.1/8.3, April–June base payment is ₹16,941.00, bonus ₹255.00 and total ₹17,196.00; regular total is ₹16,113.00.

Code screenshot

Task 12 - Code

```python
from decimal import Decimal, ROUND_HALF_UP
CENT = Decimal("0.01")
FAT_STANDARD, SNF_STANDARD = Decimal("3.5"), Decimal("8.5")
BONUS_THRESHOLD, BONUS_PER_LITRE = Decimal("500"), Decimal("0.50")
CARDS = {
    "regular": {"base": Decimal("30.00"), "fat": Decimal("0.40"), "snf": Decimal("0.30")},
    "april_june": {"base": Decimal("32.00"), "fat": Decimal("0.45"), "snf": Decimal("0.30")},
}
def money(x):
    return Decimal(str(x)).quantize(CENT, rounding=ROUND_HALF_UP)
def steps(x, standard):
    return int(((Decimal(str(x))-standard)*10).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
def rate_for(fat, snf, season="regular"):
    c=CARDS[season]
    return money(c["base"]+steps(fat,FAT_STANDARD)*c["fat"]+steps(snf,SNF_STANDARD)*c["snf"])
def settlement(entries, season="regular"):
    litres=sum((Decimal(str(l)) for l,f,s in entries),Decimal("0"))
    base=sum((money(Decimal(str(l))*rate_for(f,s,season)) for l,f,s in entries),Decimal("0.00"))
    bonus=money(litres*BONUS_PER_LITRE) if litres>BONUS_THRESHOLD else Decimal("0.00")
    return {"litres":litres,"base":base,"bonus":bonus,"total":base+bonus}
entries=[(300,4.2,8.7),(210,3.1,8.3)]
for season in ("april_june","regular"):
    print(season, settlement(entries,season))
assert rate_for(4.2,8.7,"april_june")==Decimal("35.75")
assert settlement([(250,3.5,8.5),(250,3.5,8.5)],"april_june")["bonus"]==Decimal("0.00")
assert settlement([(250,3.5,8.5),(251,3.5,8.5)],"april_june")["bonus"]==Decimal("250.50")
print("All revised design tests passed.")
```

Execution output screenshot

Task 12 - Execution output

```
$ python task12.py
april_june {'litres': Decimal('510'), 'base': Decimal('16941.00'), 'bonus': Decimal('255.00'), 'total': Decimal('17196.00')}
regular {'litres': Decimal('510'), 'base': Decimal('15858.00'), 'bonus': Decimal('255.00'), 'total': Decimal('16113.00')}
All revised design tests passed.
```